# DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC,
Accredited by NAAC with 'A' grade & ISO 9001 – 2015 Certified Institution)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560 111, India

## DEPARTMENT OF ARTIFICIAL INTELLGENCE & MACHINE LEARNING


## Alternate Assessment Tool

**Course Title: Biology for Engineers**
**Course Code: 22BE68**
**Semester: VI**


Submitted by

**Shreya V (1DS23AI055)**

Under the Guidance of
Dr Aruna M G
Professor
Department of Artificial Intelligence & Machine Learning
DSCE, Bengaluru



VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANASANGAMA, BELAGAVI-590018, KARNATAKA, INDIA
2025-26

<table>
<tr><td>Problem Statement</td><td colspan="5">Understanding DNA mutations and their impact on protein synthesis is a complex process that typically requires strong theoretical knowledge and manual analysis of genetic sequences. It becomes difficult for students and beginners to visualize how small changes in nucleotide sequences affect amino acids and overall protein structure. Additionally, identifying mutation types such as silent, missense, nonsense, and frameshift mutations manually is time-consuming and prone to errors. The DNA Mutation Analysis System addresses this by providing a computational tool that automatically analyzes DNA sequences, detects mutations, classifies them, and presents the results through an interactive and visually intuitive interface along with 3D DNA visualization for better understanding.</td></tr>
<tr><td>AAT MARKS</td><td>Design (5)</td><td>Execution (5)</td><td>Result/ Demo (5)</td><td>Innovation/ Extra Features (5)</td><td>Total</td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>TOOLS USED</td><td colspan="5">HTML, CSS, JavaScript, Web-based 3D Viewer (<model-viewer>), Bioinformatics Logic (Codon Table Mapping)</td></tr>
<tr><td>MAP only one SDG</td><td colspan="5">SDG 4: Quality Education</td></tr>
<tr><td>MAP COURSE OUTCOME of BFE</td><td colspan="5">CO1: Elucidate the basic biological concepts via relevant industrial applications and case studies.</td></tr>
<tr><td>MAP PROGRAM OUTCOME (PO)</td><td colspan="5">PO2: Problem Analysis:<br><br>Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)<br><br>PO3: Design/Development of Solutions:<br><br>Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for public health, safety, environment, and societal impact. (WK5)<br><br>PO4: Conduct Investigations of Complex Problems:</td></tr>
</table>

| | |
|---|---|
| | Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8)<br><br>**PO5: Engineering Tool Usage:**<br><br>Create, select and apply appropriate techniques, resources and modern engineering & IT tools to solve complex engineering problems. |
| MAP PROGRAM SPECIFIC OUTCOME(PSO) of DEPARTMENT | **PSO 1**: Graduates will demonstrate strong programming and problem-solving abilities for the design, development, and maintenance of software solutions.<br><br>**PSO 2**: Graduates will be able to identify, formulate, analyze, and solve real-world problems by applying principles of Artificial Intelligence and Machine Learning.<br><br>**PSO 3**: Graduates will be proficient to efficiently manage computer system resources using cloud computing technologies. |
| COURSE COORDINATOR | Dr ARUNA M G |

**Dr. Aruna M G**
Professor & Course Coordinator
Artificial Intelligence & Machine Learning
Dayananda Sagar College of Engineering
Bangalore-560111

**Dr. Vindhya P Malagi**
Head of the Department
Artificial Intelligence & Machine Learning
Dayananda Sagar College of Engineering
Bangalore-560111

# ABSTRACT

The DNA Mutation Analysis System is a web-based bioinformatics simulation tool designed to analyze genetic sequences and visualize mutation effects through an interactive interface. The system integrates molecular biology principles with computational logic to simulate DNA processing, mutation detection, and protein translation. It allows users to input DNA sequences, validates nucleotide correctness, and compares them with a reference sequence derived from the human β-globin gene. Using codon-based translation and a standard genetic code table, the system converts DNA into amino acid sequences and identifies variations at both nucleotide and protein levels.

Mutations are classified into biologically relevant categories such as silent, missense, nonsense, and frameshift mutations. The system also evaluates mutation severity and calculates mutation rates. A visually rich interface with color-coded mutation highlighting and an embedded 3D DNA model enhances understanding. This project bridges biology and computation, providing an interactive platform for learning genetic mutation analysis.

# Table of Contents


..... 2

ABSTRACT ..... 3

INTRODUCTION ..... 5

PROJECT DETAILS ..... 6

1. Data source and labels ..... 6
2. Biological principles and system logic ..... 6
3. Algorithm design and processing workflow ..... 7
4. Mutation detection and classification engine ..... 7
5. DNA to protein translation module ..... 8
6. Desktop application interface ..... 8
7. Visualization and result interpretation ..... 9
8. Input/output handling and system integration ..... 9

OBJECTIVES ..... 10

IMPLEMENTATION ..... 11

OUTPUT ..... 15

REFERENCES ..... 17

**CHAPTER 1**

# INTRODUCTION

DNA mutations are fundamental to genetic variation and play a significant role in biological processes such as evolution, disease development, and protein synthesis. Even a single change in a nucleotide sequence can lead to significant alterations in protein structure and function. However, understanding these effects through traditional methods often requires strong theoretical knowledge and manual analysis, making it challenging for beginners and students.

The DNA Mutation Analysis System addresses this challenge by providing a computational and visual platform that simulates biological processes in an interactive manner. Instead of manually analyzing sequences, users can input DNA data and instantly observe how mutations influence codons, amino acids, and overall protein structure.

At the molecular level, DNA is composed of four nucleotides—adenine, thymine, cytosine, and guanine—which follow specific base-pairing rules. The system implements these rules to generate complementary and reverse-complement strands, reflecting the antiparallel nature of DNA. Furthermore, DNA sequences are interpreted in triplets known as codons, each of which corresponds to a specific amino acid according to the standard genetic code.

The system leverages this biological framework to perform translation from DNA to protein sequences. It compares the translated output of user-provided DNA with that of a reference sequence, enabling the identification of mutations. These mutations are then classified based on their biological consequences, such as whether they alter amino acids or introduce premature stop codons.

In addition to computational analysis, the system emphasizes visualization. A modern user interface provides real-time feedback, while color-coded highlights help distinguish between normal and mutated regions. The inclusion of a 3D DNA model further enhances understanding by allowing users to interact with a visual representation of genetic structures.

Overall, this project demonstrates how computational tools can simplify complex biological concepts and make them more accessible. It highlights the interdisciplinary nature of bioinformatics by combining biology, programming, and user interface design into a cohesive system.

CHAPTER 2

# PROJECT DETAILS

## 1. Data source and labels

The system operates using a predefined reference DNA sequence derived from a fragment of the human β-globin gene. This gene is widely studied in molecular biology and is commonly used as a standard reference in mutation analysis.

The reference sequence serves as a baseline for comparison, allowing the system to identify variations in user-input DNA. From this sequence, a corresponding protein sequence is generated using codon-based translation, forming the reference protein.

Unlike large-scale bioinformatics systems that rely on extensive datasets, this project focuses on real-time analysis of user-provided sequences. This simplifies implementation while still maintaining biological accuracy and relevance.

## 2. Biological principles and system logic

The system is grounded in essential molecular biology principles:

DNA is composed of nucleotides: A, T, C, G

Base pairing follows Watson-Crick rules:

- Adenine pairs with Thymine
- Cytosine pairs with Guanine

The system generates complementary strands and reverse complements to simulate DNA's antiparallel structure. This reflects how DNA exists in nature, with strands running in opposite directions (5'→3' and 3'→5').

DNA is processed in groups of three nucleotides called codons. Each codon corresponds to a specific amino acid or a stop signal. The system uses a complete codon table to map these codons to amino acids, ensuring biological accuracy. Additionally, amino acids are

categorized based on their chemical properties (hydrophobic, polar, charged), which helps in determining mutation severity during classification.

## 3. Algorithm design and processing workflow

The system follows a structured computational workflow:

1. Accept DNA input from user
2. Validate input using pattern matching
3. Generate complementary and reverse strands
4. Divide DNA into codons
5. Translate codons into amino acids
6. Compare with reference sequence
7. Detect mismatches
8. Classify mutations
9. Calculate mutation rate
10. Display results

This workflow ensures that each step mirrors a real biological process while maintaining computational efficiency.

## 4. Mutation detection and classification engine

The mutation engine is a core component of the system. It performs base-by-base comparison between the user sequence and the reference sequence.

For each mismatch:

- The system identifies the corresponding codon
- Determines the affected amino acid
- Classifies the mutation

Mutation types include:

- Silent Mutation: No change in amino acid
- Missense Mutation: Amino acid substitution
- Conservative (similar properties)

- Non-conservative (different properties)
- Nonsense Mutation: Formation of STOP codon
- Frameshift Mutation: Caused by insertions/deletions

Severity is assigned based on biological impact, helping users understand the significance of mutations.

## 5. DNA to protein translation module

This module converts DNA sequences into protein sequences using the standard genetic code.

Key features:

- Codon-based translation
- Support for reading frames
- STOP codon detection
- Amino acid mapping

The system generates protein sequences for both the reference DNA and user input, enabling direct comparison and highlighting differences.

## 6. Desktop application interface

The system is implemented as a web-based application with a structured and interactive interface.

Features include:

- DNA input field
- Action buttons (Analyze, Clear)
- Status display messages
- Output panels for DNA and protein

The interface uses a dark theme with gradient backgrounds and glass-like UI components, enhancing visual appeal while maintaining readability.

## 7. Visualization and result interpretation

Visualization plays a crucial role in understanding results.

The system uses:

- Color-coded mutation highlighting
- Animated effects for mutated bases
- Structured display of mutation details

Each mutation is shown with:

- Position
- Base change
- Codon change
- Amino acid substitution

Additionally, a 3D DNA model is included using a web-based viewer. Users can interact with the model through rotation, zoom, and AR features, making the experience more immersive.

## 8. Input/output handling and system integration

The system ensures robust input validation and structured output generation.

Input Handling:

- Accepts DNA sequences from user
- Validates characters using regular expressions
- Ensures minimum sequence length

Output Handling:

- Displays DNA strands
- Shows protein sequences
- Highlights mutations
- Provides detailed analysis

**CHAPTER 3**

# OBJECTIVES

- Simulate DNA to protein translation using biological principles: Implements codon-based translation and the standard genetic code to convert DNA sequences into amino acid chains, demonstrating the process of protein synthesis computationally.
- Detect and classify genetic mutations accurately: Compares user DNA with a reference sequence to identify variations and categorizes them into silent, missense, nonsense, and frameshift mutations based on their biological impact.
- Provide a visual representation of mutation effects: Displays mutations using color-coded highlights and structured outputs, enabling clear understanding of nucleotide, codon, and amino acid-level changes.
- Develop an interactive bioinformatics learning tool: Creates a user-friendly platform where users can input DNA sequences and observe real-time analysis, making complex genetic concepts easier to understand.
- Integrate computational logic with biological concepts: Applies programming techniques such as validation, mapping, and comparison to model real biological processes efficiently.
- Enhance user understanding through visualization: Uses UI elements and a 3D DNA model viewer to improve conceptual clarity and engagement through interactive exploration.

**CHAPTER 4**

# IMPLEMENTATION

Following Pseudocode explain the working of the entire application.

```
BEGIN APPLICATION

Initialize UI and DNA model viewer

WHEN user enters DNA:

    Validate sequence format

    IF invalid:

        Show error

Generate reverse complement

Split DNA into codons

Translate codons into amino acids

Compare with reference:

    Identify mismatches

    Classify mutation types

    Determine severity

Calculate mutation rate

Display:

    DNA strands

    Protein sequences

    Mutation details

Render interactive 3D DNA model
```

```
END APPLICATION
```

The above pseudocode outlines the overall working logic of the DNA Mutation Analysis System in a structured and sequential manner. The application begins by initializing the user interface and loading the 3D DNA model viewer, ensuring that both the computational and visualization components are ready for interaction. When the user inputs a DNA sequence, the system first performs validation to ensure that only valid nucleotide characters (A, T, C, G) are present. If the input is invalid, an appropriate error message is displayed to guide the user. Once validated, the system generates the reverse complement of the DNA sequence to simulate the template strand, reflecting the antiparallel nature of DNA.

After preprocessing, the DNA sequence is divided into codons (triplets), which are then translated into amino acids using the standard genetic code. The translated sequence is compared with a predefined reference sequence to detect mismatches. Based on these differences, the system classifies mutation types and determines their severity. It further calculates the mutation rate to quantify the extent of variation. Finally, the results are presented to the user, including DNA strands, protein sequences, and detailed mutation analysis. The process concludes with rendering an interactive 3D DNA model, allowing users to visually explore the biological structure alongside the computed results.

In this project, a comparison is made between a rule-based approach (implemented using logical conditions and regex-style validation) and a machine learning approach using Logistic Regression to analyze DNA mutation patterns. While Logistic Regression can be used to classify mutation severity based on extracted features, the rule-based system directly applies biological principles for precise mutation identification.

The rule-based approach leverages deterministic mappings such as codon-to-amino acid relationships and predefined mutation categories, ensuring that the output remains biologically accurate and interpretable. In contrast, Logistic Regression depends on numerical feature extraction and statistical learning, which introduces abstraction between the input DNA sequence and the final prediction. This means that while Logistic Regression can generalize patterns, it may not always capture the exact biological reasoning behind a mutation. Additionally, implementing Logistic Regression requires a labeled dataset, feature engineering, and model training, which increases system complexity. For this project, where the underlying biological rules are already well-established, the rule-based method provides a more direct, efficient, and reliable solution.

## Comparison of Rule-Based System and Logistic Regression Model

| Parameter | Rule-Based System (Regex / Logic-Based) | Logistic Regression Model (ML) |
|---|---|---|
| **Approach Type** | Deterministic | Machine Learning |
| **Working Principle** | Uses predefined biological rules | Uses weighted linear combination of features |
| **Input** | Raw DNA sequence | Extracted numerical features (mutation rate, codon changes, etc.) |
| **Output** | Exact mutation type + severity (silent, missense, nonsense) | Probability-based classification (e.g., Low Risk / High Risk) |
| **Accuracy** | Very High (for known biological rules) | Moderate to High (depends on training data quality) |
| **Generalization** | Poor (cannot handle unseen or undefined patterns) | Good (can generalize to new unseen data) |
| **Flexibility** | Low (rules must be manually modified) | High (model improves with retraining) |
| **Interpretability** | Very High (direct logical mapping) | Moderate (coefficients explain influence but not exact reasoning) |
| **Training Required** | No | Yes (requires labeled dataset) |
| **Execution Time** | Very Fast (direct computation) | Fast (but includes model inference overhead) |

| | | |
|---|---|---|
| **Data Requirement** | No dataset needed | Requires structured training dataset |
| **Error Handling** | Limited to predefined conditions | Can adapt based on learned patterns |
| **Scalability** | Low | High |
| **Real-world Adaptability** | Limited | Better for large-scale predictive systems |

Although Logistic Regression is a powerful machine learning technique capable of identifying patterns and making predictions from data, it is not the most suitable approach for this specific application of DNA mutation analysis. The primary reason lies in the nature of the problem itself. DNA mutation classification is governed by well-defined biological rules, such as codon-to-amino acid mapping and established mutation categories (silent, missense, nonsense). These rules are deterministic and do not require probabilistic approximation.

The rule-based system, implemented using direct logical conditions and validation techniques similar to regex, provides exact and biologically accurate results. It does not rely on training data, thereby eliminating the risk of model bias, overfitting, or incorrect predictions due to insufficient or noisy datasets. In contrast, Logistic Regression requires carefully engineered features, labeled training data, and proper tuning to achieve reliable performance. Even then, it can only provide probability-based outputs rather than exact mutation classifications.

Additionally, the rule-based approach is significantly faster and more interpretable. Each output can be directly traced back to a biological rule, making the system transparent and easy to understand. Logistic Regression, while interpretable to some extent, does not provide the same level of clarity in explaining why a specific mutation classification was made.

Furthermore, for small-scale and educational applications like this project, the overhead of training and maintaining a machine learning model is unnecessary. The rule-based system is lightweight, efficient, and sufficient for accurately modeling the problem domain.

Therefore, while Logistic Regression offers better scalability and generalization in data-driven environments, the rule-based approach is more appropriate for this project due to its accuracy, simplicity, speed, and direct alignment with biological principles.

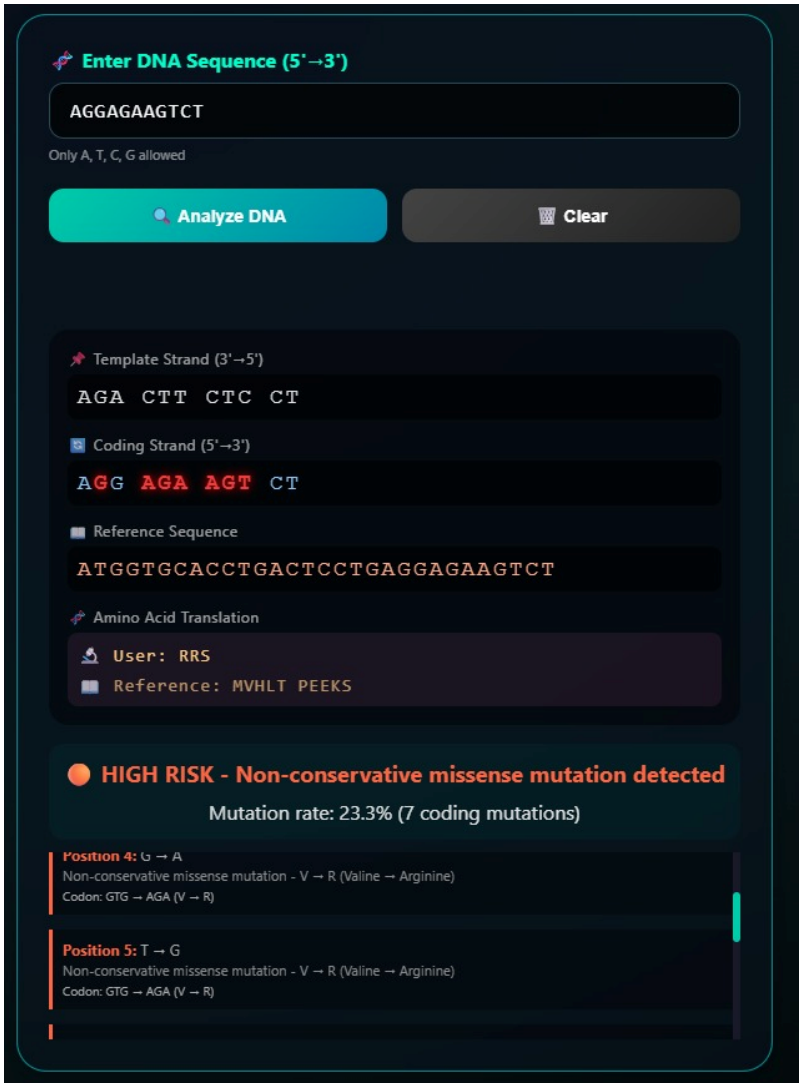
## CHAPTER 5

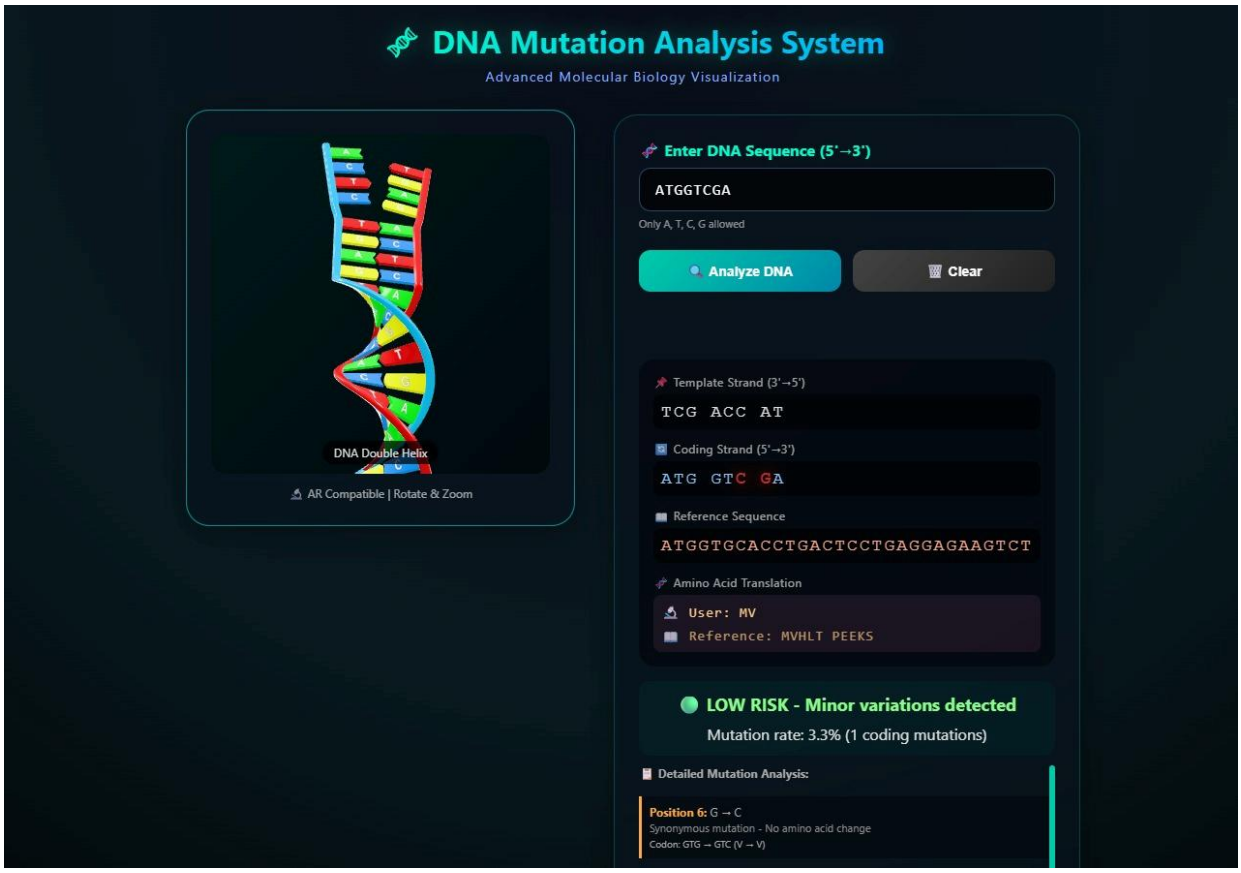
# OUTPUT

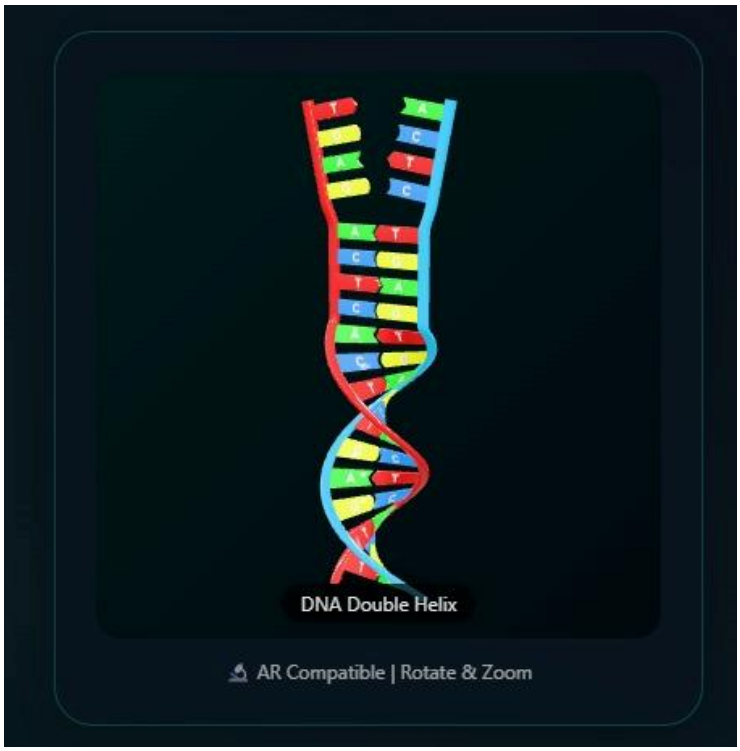
The system produces multiple outputs that help users understand mutation effects clearly:

- DNA coding and template strands
- Codon-wise breakdown
- Amino acid sequence
- Mutation classification
- Severity level and mutation rate
- Detailed mutation descriptions

The results are displayed in a structured and visually intuitive format, making it easier to interpret complex biological data.




DNA Mutation Analysis System
Advanced Molecular Biology Visualization
DNA Double Helix
AR Compatible | Rotate & Zoom
Enter DNA Sequence (5'→3')
ATGGTCGA
Only A, T, C, G allowed
Analyze DNA
Clear
Template Strand (3'→5')
TCG ACC AT
Coding Strand (5'→3')
ATG GTC GA
Reference Sequence
ATGGTGCACCTGACTCCTGAGGAGAAGTCT
Amino Acid Translation
User: MV
Reference: MVHLT PEEKS
LOW RISK - Minor variations detected
Mutation rate: 3.3% (1 coding mutations)
Detailed Mutation Analysis:
Position 6: G → C
Synonymous mutation - No amino acid change
Codon: GTG → GTC (V → V)


DNA Double Helix
AR Compatible | Rotate & Zoom

# REFERENCES


1. NCBI – National Center for Biotechnology Information

https://www.ncbi.nlm.nih.gov/

2. Standard Genetic Code Table (NCBI)

https://www.ncbi.nlm.nih.gov/Taxonomy/Utils/wprintgc.cgi

3. Molecular Biology of the Cell – Alberts et al. (Reference Book)

https://www.ncbi.nlm.nih.gov/books/NBK21054/

4. Khan Academy – DNA, RNA, and Protein Synthesis

https://www.khanacademy.org/science/ap-biology/gene-expression-and-regulation

5. Nature Education – Scitable (Genetics & Mutation Concepts)

https://www.nature.com/scitable/topicpage/mutations-1127/

6. Google Model Viewer Documentation (3D Web Visualization)

https://modelviewer.dev/

7. Introduction to Bioinformatics – Algorithms and Applications

https://www.ebi.ac.uk/training/online/course/introduction-bioinformatics